PRACTICA 4 –

1.- Implementar una clase ColoreoGrafo.java, de modo que con la clase proporcionada Devorador.java podamos calcular una solución y visualizarla con el módulo proporcionado en Python. Se debe añadir el JAR provisto al buildpath para que este último funcione.

Razone convenientemente la complejidad temporal del algoritmo implementado.

Implementar una clase DevoradorTiempos.java, que emplee los grafos contenidos en sols y calculando el tiempo que tarda el algoritmo hecho en el apartado anterior en resolver el problema, de forma que se pueda ir rellenando la siguiente tabla.

Razone: ¿siguen la complejidad calculada los tiempos de la tabla

```java
public class ColoreoGrafo {

    // se busca utilizar el heurístico de minimo de colores posibles
    public static Map<String, String> realizarVoraz(Map<String, List<String>> grafo) {
        Map<String, String> mapaColores = new HashMap<String, String>();
        String[] colores = {"red", "blue", "green",
                            "yellow", "orange", "purple", "cyan", "magenta", "lime"};


        for (Entry<String, List<String>> nodo : grafo.entrySet()) {
            String key = nodo.getKey();

            if (!mapaColores.containsKey(key)) {
                List<String> nodosAdyacentes = nodo.getValue();
                int highestColor = 0;

                for (String nodoAdyacente : nodosAdyacentes) {
                    int color = 0;

                    if (mapaColores.containsKey(nodoAdyacente)) {
                        String colorString = mapaColores.get(nodoAdyacente);

                        while (!(colores[color]==colorString)) {
                            color++;
                        }
                        color++;
                    }
                    if (color>highestColor) {
                        highestColor = color;
                    }

                }

                mapaColores.put(key, colores[highestColor]);
            }
        }



        return mapaColores;
    }

}
```

La complejidad del algoritmo es $O(n^2)$, escalando con el número de nodos y vecinos de cada nodo. Tiene otro bucle interno, recorriendo la matriz de colores, pero eso es un bucle de en el peor de los casos O(7), ya que la paleta de colores esta hard-codeada y no escala con el tamaño del problema.

Tristemente, no he podido ejecutar el código, ya que esté se niega a reconocer las clases como existentes por motivos que desconozco.

2.- Implementar el módulo coloreado_grafo.py, de modo que podamos calcular una solución y visualizarla con el módulo proporcionado en Python. Razone convenientemente la complejidad temporal del algoritmo implementado. Implementar un módulo devorador_tiempos.py, que emplee los grafos contenidos en sols y calculando el tiempo que tarda el algoritmo hecho en el apartado anterior en resolver el problema, de forma que se pueda ir rellenando la siguiente tabla. Razone: ¿siguen la complejidad calculada los tiempos de la tabla?

```python
def realizar_voraz(mapa : dict):
    colores = ["red", "blue", "green", "yellow", "orange", "purple", "cyan", "magenta", "lime"]
    solucion = dict()

    nodeList = bfs(mapa, 0)

    for key in nodeList:
        key = str(key)
        color = 0

        if key not in solucion:
            lista = mapa.get(key)

            availableColors = returnColorList(lista,colores,solucion)
            color = 0
            for i in range(len(availableColors)):
                if (availableColors[i]==False):
                    color = i
                    break
            if (color>=len(colores)):
                color=0
            solucion.update({key: colores[color]})

    return solucion
```

```python
def returnColorList(lista, colores, solucion):
    visited = [False] * len(colores)
    colores2 = []
    for entry in lista:
        entry = str(entry)
        if entry in solucion:
            colores2.append(solucion.get(entry))

    i = 0
    for color in colores2:
        i = 0
        while i<len(colores):
            if color == colores[i]:
                if(visited[i]==False):
                    visited[i]=True
            i+=1


    return visited
```

```python
def bfs(graph, start):
    visited = [False] * len(graph)
    queue = []
    queue.append(start)
    visited[start] = True
    order = []

    while queue:
        start = queue.pop(0)
        start = str(start)
        order.append(start)

        for i in graph.get(start):
            i = int(i)
            if not visited[i]:
                queue.append(i)
                visited[i] = True


    return order
```

El algoritmo principal, realizar_voraz, tiene dos bucles for, pero su complejidad nunca es de $O(n^2)$ (POR SI SOLO), esto se debe a que el segundo bucle for anidado recorre la lista de colores, la cual tiene un tamaño fijo.

Este, sin embargo, tiene un método interno y un método externo; returnColorList y BFS

El primero busca en la lista de adyacentes de un nodo (que puede ser n) y busca los colores correspondientes de cada uno; después, recorre un vector de colores "visitados" (que guarda que colores aparecen con un booleano) y lo devuelve al método principal. Así recorriendo en orden la lista, se puede saber cuál es el primer color que queda disponible. (el primero que tenga False).

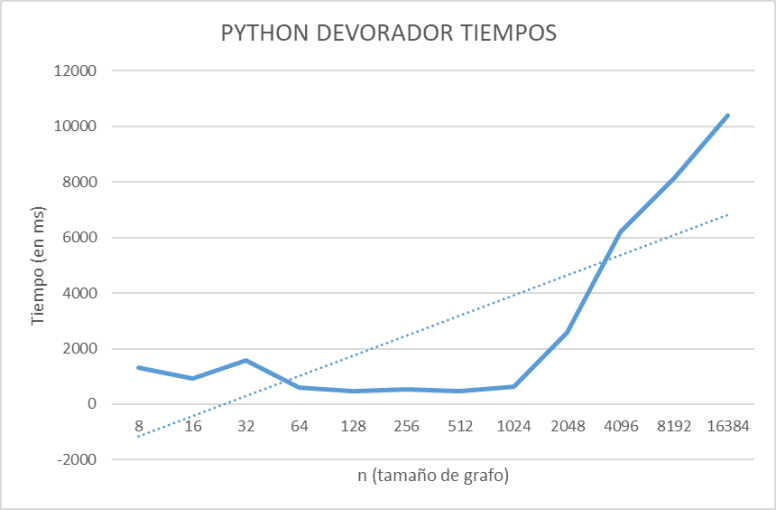
El segundo es el algoritmo de BFS, Breathd-First Search (o búsqueda de anchura), que recorre los nodos desde el inicio, después sus adyacentes, después los adyacentes del primer adyacente, y así, sin repetir nodos. Se utiliza para darle a un orden al coloreo de nodos antes de empezar.

¿Cuál es su complejidad entonces?

Bueno, la complejidad del método de BFS es de O(n), donde n es la suma entre |V| + |E|, V siendo el número de vértices y E siendo el número de aristas y la de returnColorList es O(n+n), siendo la segunda n el número de colores que se hayan hardcodeado.

Por tanto, teóricamente, la complejidad de este algoritmo, es de $O(n^2)$. Sin embargo, este es el peor caso, en el que TODOS los vértices están coloreados. En el mejor caso, ningún vértice está coloreado, por lo que nunca entra en el bucle, que lo dejaría con una complejidad de O(n).

n	tiempo
8	1326
16	939
32	1581
64	610
128	459
256	525
512	483
1024	639
2048	2586
4096	6187
8192	8171
16384	10394
32768	COLGADO
65536	COLGADO



La tabla de tiempos se comporta extremadamente rara por esto, depende del número de nodos vértices etc. Además, la representación gráfica colgó el programa.